

Automobile Inheritance Hierarchy

Java OOP Assignment — Wrapper Classes, Inheritance,
Polymorphism, Static/Dynamic Binding, and Nested Classes

Class hierarchy: Automobile → Toyota → { Corolla, Tundra, Highlander }
Highlander contains a nested class ThirdRowSeating (7 or 8 seat configuration).

Overview

This program models a small automobile inheritance hierarchy in Java. **Automobile** is the base class and declares four instance variables using Wrapper classes only (Integer year, Double price, String slogan, Character type) with full getters/setters. **Toyota** extends Automobile and fixes the brand defaults (year=1937, price=0.0, slogan="Let's Go Places", type='n'). **Corolla**, **Tundra**, and **Highlander** each extend Toyota, set their own year/price/type, and deliberately leave the slogan un-overridden so it is inherited unchanged from Toyota.

Each class in the hierarchy overrides **sound()** to print a distinct sound. **Corolla** adds an extra method, **printFuelEconomy()**, that does not exist on any other class. **Highlander** contains a **public static nested class**, **ThirdRowSeating**, that lets a caller configure the third row as 7 or 8 seats (with validation). The **Main** class creates objects of every class and demonstrates: autoboxing/unboxing of the Wrapper classes, dynamic binding (overridden sound() resolved at runtime through an Automobile reference), static binding (inherited, non-overridden getters resolved at compile time), the Corolla-only method, and instantiation/use of the Highlander nested class.

Source Code

Automobile.java

```
/**
 * Base class for the inheritance hierarchy. All instance variables use
 * Wrapper classes (Integer, Double, String, Character) instead of primitives.
 */
public class Automobile {

    private Integer year;
    private Double price;
    private String slogan;
    private Character type;

    public Automobile() {
    }

    public Automobile(Integer year, Double price, String slogan, Character type) {
        this.year = year;
        this.price = price;
        this.slogan = slogan;
        this.type = type;
    }

    public Integer getYear() {
        return year;
    }

    public void setYear(Integer year) {
        this.year = year;
    }

    public Double getPrice() {
        return price;
    }

    public void setPrice(Double price) {
        this.price = price;
    }

    public String getSlogan() {
        return slogan;
    }

    public void setSlogan(String slogan) {
        this.slogan = slogan;
    }

    public Character getType() {
        return type;
    }

    public void setType(Character type) {
        this.type = type;
    }

    public void sound() {
        System.out.println("Automobile sound: Vroom!");
    }

    @Override
    public String toString() {
```

```

        return getClass().getSimpleName() + " [year=" + year + ", price=" + price
            + ", slogan=\"" + slogan + "\", type=" + type + "]\n";
    }
}

```

Toyota.java

```

/**
 * Toyota inherits from Automobile and fixes the brand-level defaults.
 */
public class Toyota extends Automobile {

    public Toyota() {
        super(1937, 0.0, "Let's Go Places", 'n');
    }

    protected Toyota(Integer year, Double price, String slogan, Character type) {
        super(year, price, slogan, type);
    }

    @Override
    public void sound() {
        System.out.println("Toyota sound: Reliable engine hum.");
    }
}

```

Corolla.java

```

/**
 * Corolla inherits from Toyota. The slogan is intentionally left
 * un-overridden, so it keeps Toyota's "Let's Go Places".
 */
public class Corolla extends Toyota {

    public Corolla() {
        super(2023, 21700.0, "Let's Go Places", 'c');
    }

    @Override
    public void sound() {
        System.out.println("Corolla sound: Quiet, efficient purr.");
    }

    // Method unique to Corolla - not present on any other class in the hierarchy.
    public void printFuelEconomy() {
        System.out.println("Corolla fuel economy: up to 35 city / 41 highway MPG.");
    }
}

```

Tundra.java

```

/**
 * Tundra inherits from Toyota. The slogan is intentionally left
 * un-overridden, so it keeps Toyota's "Let's Go Places".
 */
public class Tundra extends Toyota {

    public Tundra() {
        super(2023, 38965.0, "Let's Go Places", 't');
    }

    @Override
    public void sound() {
        System.out.println("Tundra sound: Deep, roaring V8 rumble.");
    }
}

```

```
}
```

Highlander.java

```
/**
 * Highlander inherits from Toyota. The slogan is intentionally left
 * un-overridden, so it keeps Toyota's "Let's Go Places".
 */
public class Highlander extends Toyota {

    public Highlander() {
        super(2023, 36620.0, "Let's Go Places", 's');
    }

    @Override
    public void sound() {
        System.out.println("Highlander sound: Smooth, confident growl.");
    }

    /**
     * Nested class letting a Highlander buyer pick a 7- or 8-seat
     * third-row configuration.
     */
    public static class ThirdRowSeating {

        private Integer seatCount;

        public ThirdRowSeating(Integer seatCount) {
            setSeatCount(seatCount);
        }

        public Integer getSeatCount() {
            return seatCount;
        }

        public void setSeatCount(Integer seatCount) {
            if (seatCount != 7 && seatCount != 8) {
                throw new IllegalArgumentException("Third row seating must be configured as 7 or 8 seats.");
            }
            this.seatCount = seatCount;
        }

        @Override
        public String toString() {
            return "Highlander configured with " + seatCount + " total seats"
                + (seatCount == 8 ? " (8-seat third row)." : " (7-seat third row).");
        }
    }
}
```

Main.java

```
import java.util.ArrayList;
import java.util.List;

/**
 * Demonstrates object creation, static and dynamic binding, wrapper
 * class usage, and the Highlander nested class.
 */
public class Main {

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println(" WRAPPER CLASS DEMONSTRATION");
    }
}
```

```

System.out.println("=====");
Integer boxedYear = 1937;           // autoboxing int -> Integer
Double boxedPrice = 21700.0;        // autoboxing double -> Double
Character boxedType = 'c';          // autoboxing char -> Character
int unboxedYear = boxedYear;        // unboxing Integer -> int
System.out.println("Integer wrapper value: " + boxedYear);
System.out.println("Double wrapper value: " + boxedPrice);
System.out.println("Character wrapper value: " + boxedType);
System.out.println("Unboxed back to int: " + unboxedYear);
System.out.println("Integer.compare(2023, 1937) = " + Integer.compare(2023, 1937));
System.out.println();

System.out.println("=====");
System.out.println(" OBJECT CREATION");
System.out.println("=====");
Automobile automobile = new Automobile(1900, 500.0, "Generic Motors Co.", 'g');
Toyota toyota = new Toyota();
Corolla corolla = new Corolla();
Tundra tundra = new Tundra();
Highlander highlander = new Highlander();

System.out.println(automobile);
System.out.println(toyota);
System.out.println(corolla);
System.out.println(tundra);
System.out.println(highlander);
System.out.println();

System.out.println("=====");
System.out.println(" DYNAMIC BINDING (runtime polymorphism)");
System.out.println("=====");
System.out.println("Each object is stored as an Automobile reference; sound() still");
System.out.println("resolves to the actual runtime type's override:");
List<Automobile> fleet = new ArrayList<>();
fleet.add(automobile);
fleet.add(toyota);
fleet.add(corolla);
fleet.add(tundra);
fleet.add(highlander);

for (Automobile a : fleet) {
    a.sound(); // dynamic (late) binding - decided at runtime by the JVM
}
System.out.println();

System.out.println("=====");
System.out.println(" STATIC BINDING (compile-time resolution)");
System.out.println("=====");
Automobile refAsAutomobile = corolla;
// getYear()/getSlogan() are inherited, not overridden anywhere in the
// hierarchy, so the compiler statically binds the call to
// Automobile's implementation regardless of the object's real type.
System.out.println("Static (declared) type: Automobile");
System.out.println("Runtime (actual) type: " + corolla.getClass().getSimpleName());
System.out.println("refAsAutomobile.getYear() -> " + refAsAutomobile.getYear());
System.out.println("refAsAutomobile.getSlogan() -> " + refAsAutomobile.getSlogan());
System.out.println();

System.out.println("=====");
System.out.println(" METHOD UNIQUE TO COROLLA");
System.out.println("=====");
corolla.printFuelEconomy();
System.out.println();
System.out.println("=====");

```

```
System.out.println(" NESTED CLASS: Highlander.ThirdRowSeating");
System.out.println("=====");
Highlander.ThirdRowSeating eightSeatConfig = new Highlander.ThirdRowSeating(8);
Highlander.ThirdRowSeating sevenSeatConfig = new Highlander.ThirdRowSeating(7);
System.out.println(eightSeatConfig);
System.out.println(sevenSeatConfig);

eightSeatConfig.setSeatCount(7);
System.out.println("After reconfiguring: " + eightSeatConfig);
}
}
```

Program Output — Demonstration

The screenshot below shows the compiled program (`javac *.java && java Main`) running end-to-end, demonstrating wrapper class autoboxing/unboxing, object creation for every class, dynamic binding of the overridden `sound()` method, static binding of inherited getters, the Corolla-only method, and the Highlander nested class.

Terminal — java Main

```
$ javac *.java && java Main
```

```
=====
WRAPPER CLASS DEMONSTRATION
=====
```

```
Integer wrapper value: 1937
Double wrapper value: 21700.0
Character wrapper value: c
Unboxed back to int: 1937
Integer.compare(2023, 1937) = 1
```

```
=====
OBJECT CREATION
=====
```

```
Automobile [year=1900, price=500.0, slogan="Generic Motors Co.", type=g]
Toyota [year=1937, price=0.0, slogan="Let's Go Places", type=n]
Corolla [year=2023, price=21700.0, slogan="Let's Go Places", type=c]
Tundra [year=2023, price=38965.0, slogan="Let's Go Places", type=t]
Highlander [year=2023, price=36620.0, slogan="Let's Go Places", type=s]
```

```
=====
DYNAMIC BINDING (runtime polymorphism)
=====
```

```
Each object is stored as an Automobile reference; sound() still
resolves to the actual runtime type's override:
Automobile sound: Vroom!
Toyota sound: Reliable engine hum.
Corolla sound: Quiet, efficient purr.
Tundra sound: Deep, roaring V8 rumble.
Highlander sound: Smooth, confident growl.
```

```
=====
STATIC BINDING (compile-time resolution)
=====
```

```
Static (declared) type: Automobile
Runtime (actual) type: Corolla
refAsAutomobile.getYear() -> 2023
refAsAutomobile.getSlogan() -> Let's Go Places
```

```
=====
METHOD UNIQUE TO COROLLA
=====
```

```
Corolla fuel economy: up to 35 city / 41 highway MPG.
```

```
=====
NESTED CLASS: Highlander.ThirdRowSeating
=====
```

```
Highlander configured with 8 total seats (8-seat third row).
Highlander configured with 7 total seats (7-seat third row).
After reconfiguring: Highlander configured with 7 total seats (7-seat third row).
```